第三章 栈和队列



通常称，栈和队列是限定**插入和删除只能**在表的“**端点**”进行的线性表。

线性表	**栈**	**队列**
Insert(L, i, x) $1 \le i \le n+1$	Insert(S, n+1, x)	Insert(Q, n
Delete(L, i) $1 \le i \le n$	Delete(S, n)	Delete(Q,

栈和队列是两种常用的数据类型

3.1 栈的类型定义

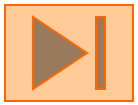
3.2 栈类型的实现

3.3 栈的应用举例

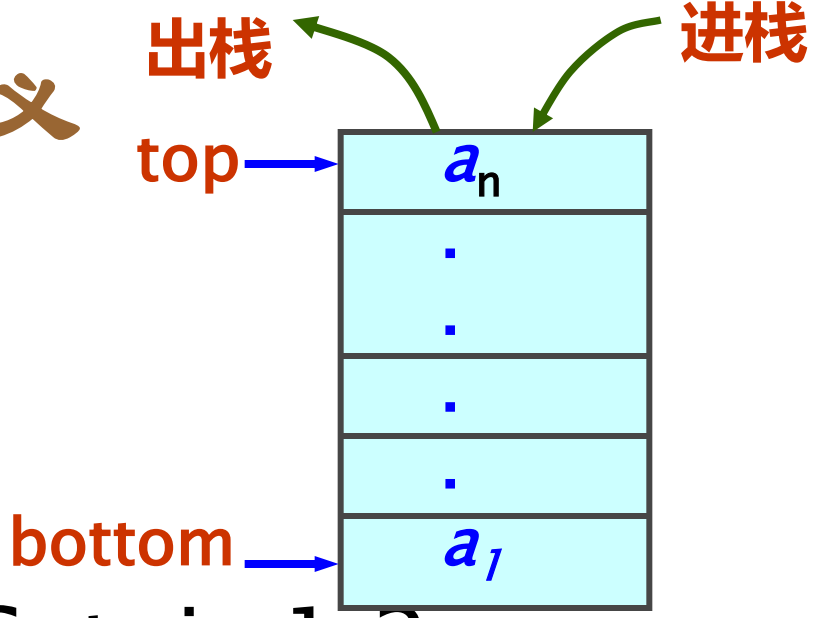
3.4 队列的类型定义

3.5 队列类型的实现

3.6 队列的应用举例



3.1 栈的类型定义



ADT Stack {

数据对象:

D = { a_i | a_i ∈ ElemSet, i = 1,2,...,n, n≥

数据关系:

R1 = { <a_{i-1}, a_i > | a_{i-1}, a_i ∈ D, i = 2,...,n

约定a_n 端为栈顶，a_1 端为栈底。

基本操作:

} ADT Stack

InitStack(&S)

DestroyStack(&S)

StackLength(S)

StackEmpty(s)

GetTop(S, &e)

ClearStack(&S)

Push(&S, e)

Pop(&S, &e)

StackTravers(S, visit())



InitStack(&S)

操作结果：构造一个空栈 S。

DestroyStack(&S)

初始条件：栈 S 已存在。

操作结果：栈 S 被销毁。

StackEmpty(S)

初始条件：栈 S 已存在。

操作结果：若栈 S 为空栈，则返回 TRUE，否则 FALSE。

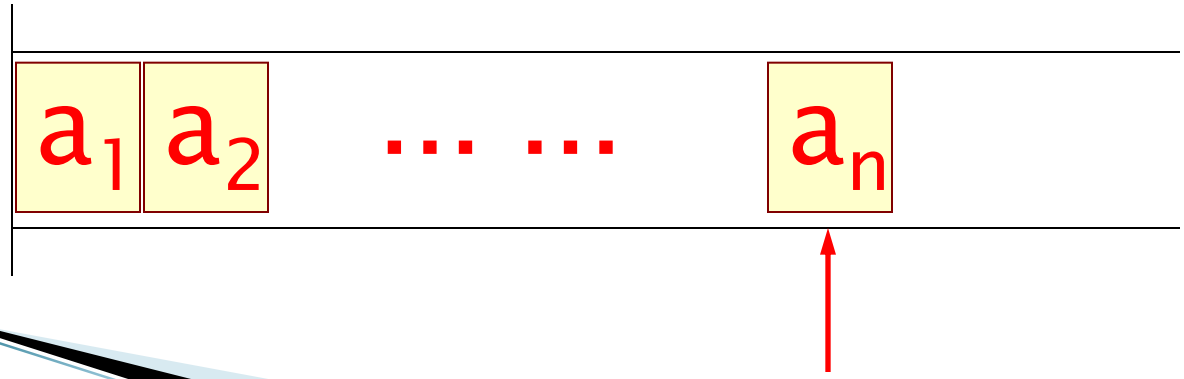
StackLength(S)

初始条件：栈 S 已存在。

操作结果：返回 S 的元素个数，即栈的长度。

GetTop(S, &e)

初始条件：栈 S 已存在且非空。**操作结果**：用 e 返回 S 的栈顶元素。



ClearStack(&S)

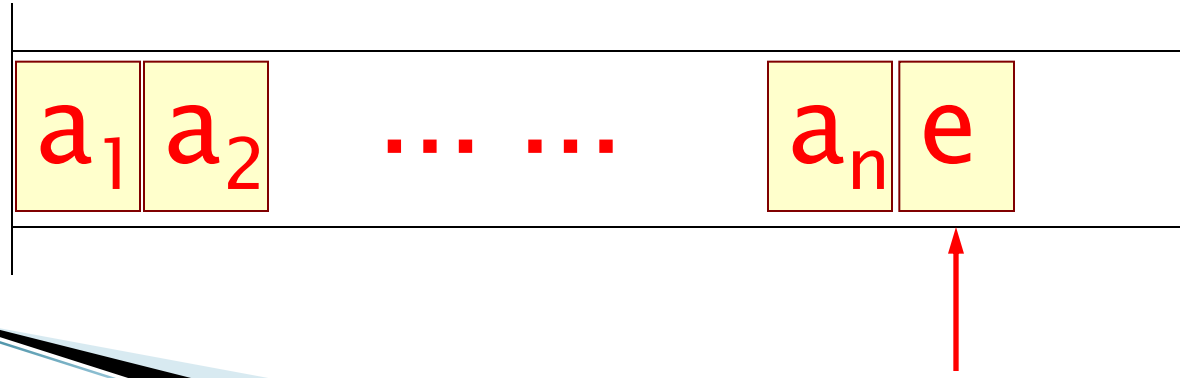
初始条件：栈 S 已存在。

操作结果：将 S 清为空栈。

Push(&S, e)

初始条件：栈 S 已存在。

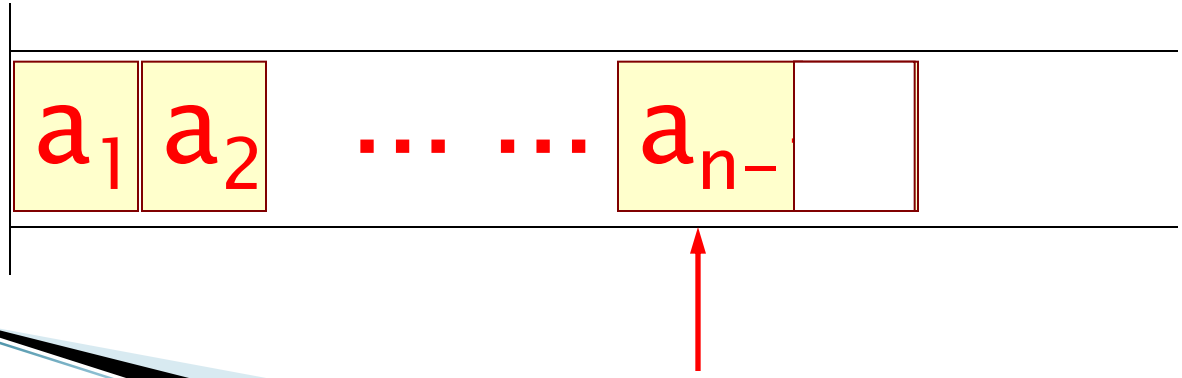
操作结果：插入元素 e 为新的栈顶元素。



Pop(&S, &e)

初始条件：栈 S 已存在且非空。

操作结果：删除 S 的栈顶元素，并用 e 返回其值。



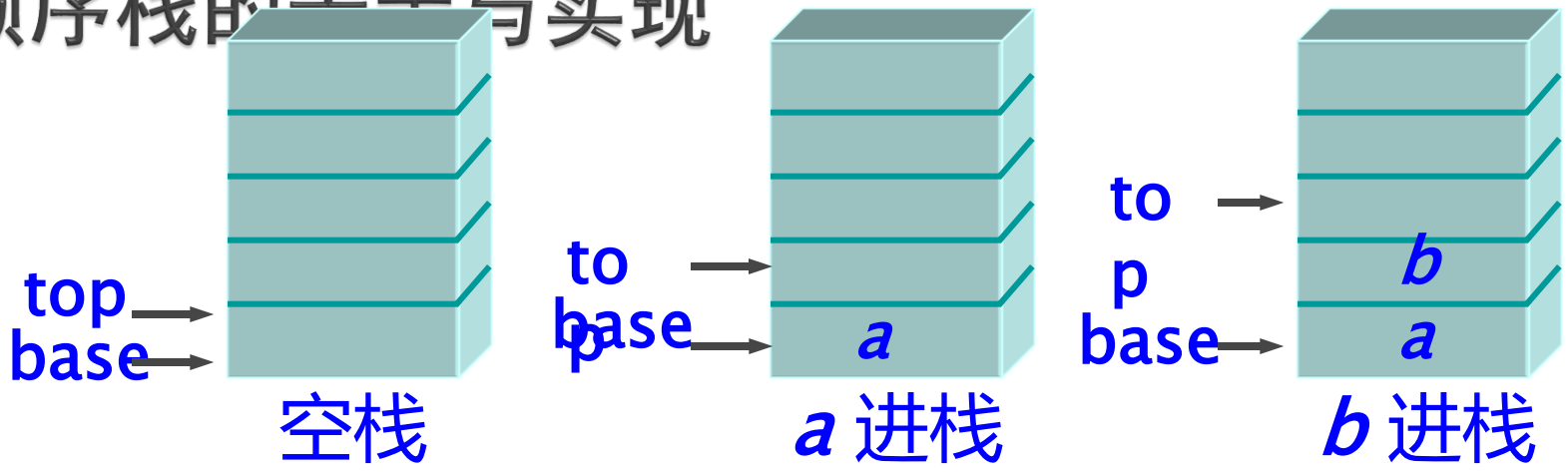
3.2 栈类型的实现

顺序栈

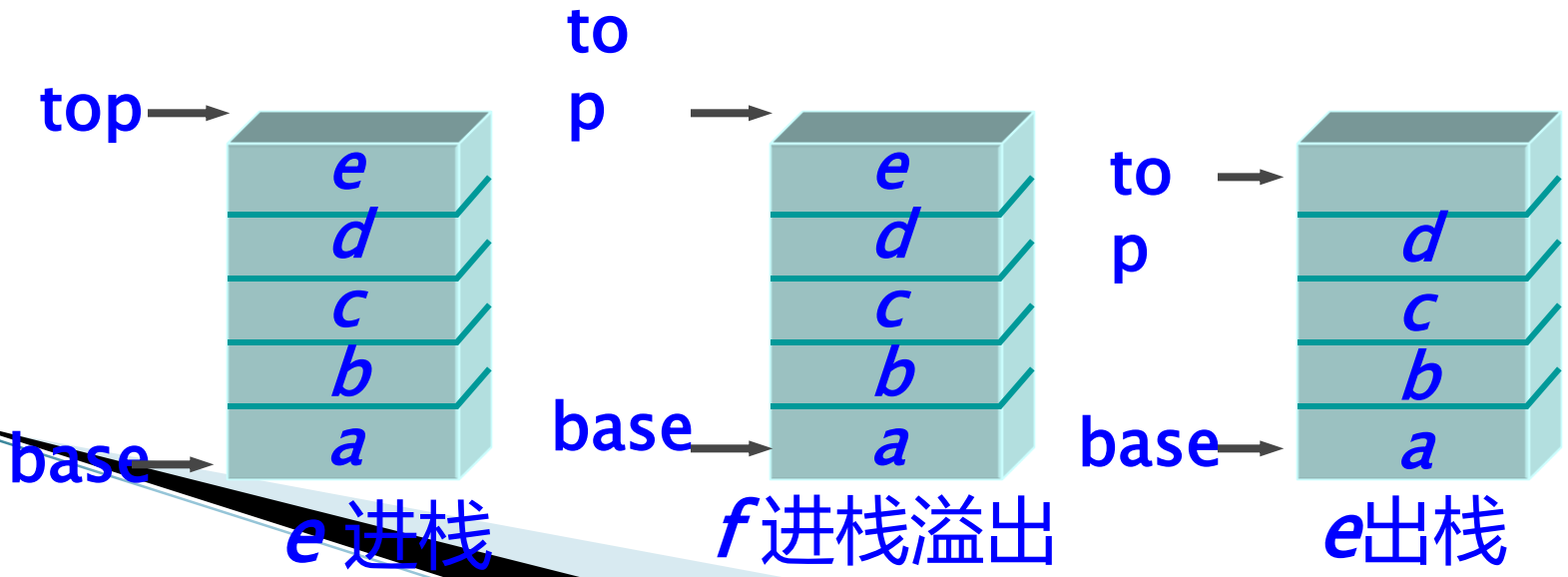
链栈



顺序栈的表示与实现



约定：top指向栈顶元素的下一个位置



类似于线性表的顺序映象实现，指向表尾的指针可以作为栈顶指针。

```
//----- 栈的顺序存储表示 -----
#define  STACK_INIT_SIZE  100;
#define  STACKINCREMENT   10;
typedef struct {
  SElemType  *base;
  SElemType  *top;
  int  stacksize;
} SqStack;
```

```
Status InitStack (SqStack &S)
{// 构造一个空栈S

S.base=(ElemType*)malloc(STACK_INIT_SIZE*

sizeof(ElemType));
   if (!S.base) exit (OVERFLOW); //存储分配失败
```

```
Status Push (SqStack &S, SElemType e) {
  if (S.top - S.base >= S.stacksize) {//栈满，追加存储空间
    S.base = (ElemType *) realloc ( S.base,
            (S.stacksize + STACKINCREMENT) *
                                    sizeof (ElemType));
    if (!S.base) exit (OVERFLOW); //存储分配失败
    S.top = S.base + S.stacksize; // 扩容后按新基址恢复栈顶
    S.stacksize += STACKINCREMENT;
  }
```

```
Status Pop (SqStack &S, SElemType &e)
{
    // 若栈不空，则删除S的栈顶元素，
    // 用e返回其值，并返回OK；
    // 否则返回ERROR
   if (S.top == S.base) return ERROR;
   e = *--S.top; // 先回退 top，再读取元素
   return OK;
}
```



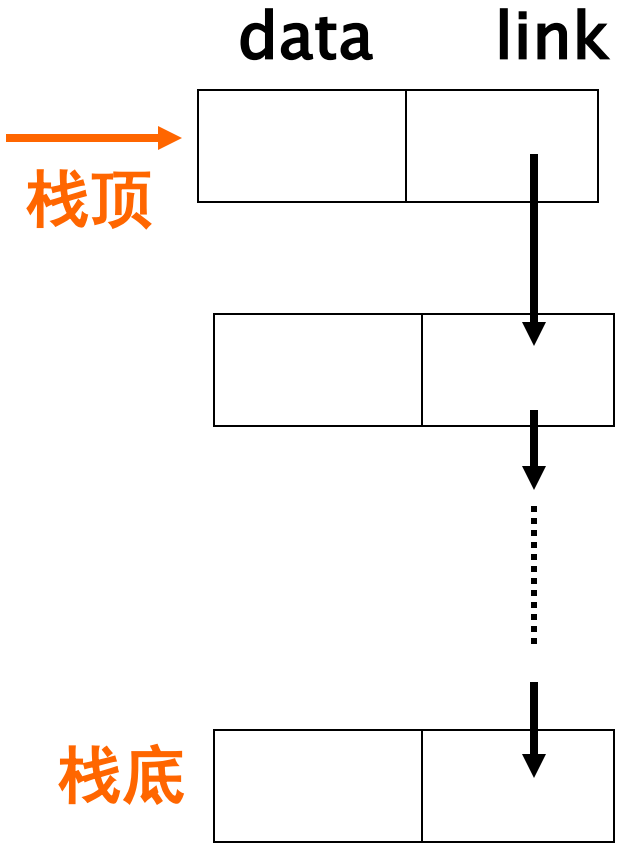
链栈

- 栈的链式存储结构称为链栈。

- 链栈的特点---

（1）插入和删除（进栈/退栈）仅在表头位置上（栈顶）进行。

（2）不需附加头结点，栈顶指针就是链表（即链栈）的头指针。



栈的链接存储结构

链栈的结点定义:

```
typedef  char  SElemType ;
typedef struct LinkNode{
        SElemType  data;
        struct LinkNode  *next;
} LinkNode,  *LinkStack;
LinkStack  top;
```

链栈的进栈算法:

```
Status Push( LinkStack &top,  SElemType  e ){
    LinkStack  p;
     p=(LinkNode *)malloc(sizeof(LinkNode));
     if ( !p ) exit(ERROR);
     p->data = e;
     p->next=top;
     top = p;
}
```

链栈的出栈算法:

```
Status Pop(LinkStack &top,  SElemType  &e){
        if ( top == NULL)
              return  ERROR;
         else {
              e = top->data;
              p= top;
              top = top->next;
              free(p);
              return OK;
        }
}
```

3.3 栈的应用举例

例一、 数制转换

例二、 括号匹配的检验

例三、 行编辑程序问题

例四、 迷宫求解

例五、 表达式求值

例六、 实现递归



例一、 数制转换

算法基于原理：

$$N = (N \text{ div } d) \times d + N \bmod d$$

例如：$(1348)_{10} = (2504)_8$，其运算过程如下：

N	N div 8	N mod 8
1348	168	4
168	21	0
21	2	5
2	0	2

计算顺序

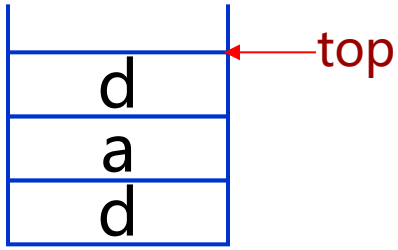
输出顺序

```
void conversion () {
    InitStack(S);
    scanf ("%d",N);
    while (N) {
      Push(S, N % 8); // 先得到的余数是低位
      N = N/8;
    }
    while (!StackEmpty(S)) {
      Pop(S,e);
      printf ( "%d", e );
    }
} // conversion
```



回文游戏

- 顺读与逆读字符串一样（不含空格）



字符串：“madam im adam”

1.读入字符串
2.去掉空格（原串）
3.压入栈
4.原串字符与出栈字符依次比较
若不等，非回文
若直到栈空都相等，回文

例二、 括号匹配的检验

假设在表达式中

（ [] （）） 或 [（ [ ] [ ] ）]

等为正确的格式，

[（] ） 或 （ [（ ） ） 或 （ （）]

）

均为不正确的格式。

检验括号是否匹配的方法可用

“期待的急迫程度”这个概念来描述。

例如：考虑下列括号序列：

[([ ] [ ])]

1 2 3 4 5 6 7 8

分析可能出现的不匹配的情况：

- 到来的右括弧并非是所“期待”的
- 到来的是“不速之客”；
- 直到结束，也没有到来所“期待”的括弧。

算法的设计思想：

1）凡出现**左括弧**，则**进栈**；

2）凡出现**右括弧**，首先检查栈是否空

若**栈空**，则表明该“**右括弧**”**多余**，

否则**和栈顶元素**比较，

若**相匹配**，则“**左括弧出栈**”，

否则表明**不匹配**。

3）**表达式**检验**结束时**，

若**栈空**，则表明表达式中**匹配正确**，

否则表明“**左括弧**”**有余**。

```
Status matching(string& exp) {
  int state = 1;
  while (i<=Length(exp) && state) {
     switch of exp[i] {
       case 左括弧:{Push(S,exp[i]); i++;
break;}
       case")": {
          if(NOT
StackEmpty(S)&&GetTop(S)="("
            {Pop(S,e);  ii++;}
          else {state = 0;}
          break;  }   … …
  }
```

例三、行编辑程序问题

如何实现?

“每接受一个字符即存入存储器” ?

并不恰当!

在用户输入一行的过程中，允许用户输入出差错，并在发现有误时可以及时更正。

合理的作法是：

设立一个输入缓冲区，用以接受用户输入的一行字符，然后逐行存入用户数据区，并假设“#”为退格符，“@”为退行符。

假设从终端接受了这样两行字符：

```
whli##ilr#e (s#*s)
    outcha@putchar(*s=#++);
```

则实际有效的是下列两行：

```
while (*s)
    putchar(*s++);
```

```
while (ch != EOF) { //EOF为全文结束符
```

```
while (ch != EOF && ch != '\n') {
  switch (ch) {
   case '#' : Pop(S, c); break;
   case '@': ClearStack(S); break;// 重置S为空
   default : Push(S, ch);  break;
  }
  ch = getchar();  // 从终端接收下一个字符
}
```

将从栈底到栈顶的字符传送至调用过程的数据区；

```
ClearStack(S);       // 重置S为空栈
if (ch != EOF)  ch = getchar();
```



例四、 迷宫求解

#	#	#	#	#	#
#		#			#
#					#
#	#			#	#
#		#		⊖	#
#	#	#	#	#	#

下一步

计算机解迷宫时，通常用的是“穷举求解”的方法：

- 从入口出发，顺某一方向向前探索，若能走通，则继续往前走；
- 否则沿**原路退回**，换一个方向再继续探索，直至所有可能的通路都探索到为止。
- 如果所有可能的通路都试探过，还是不能走到终点，那就说明该迷宫不存在从起点到终点的通道。

回溯法

求迷宫路径算法的基本思想是：

- 若当前位置“可通”，则纳入路径，继续前进；

- 若当前位置“不可通”，则后退，换方向继续探索；
- 若四周“均无通路”，则将当前位置从路径中删除出去。

求迷宫中一条从入口到出口的路径的算法

设定当前位置的初值为入口位置；

do {

若当前位置可通，

则 {将当前位置**插入栈顶**；

若该位置是出口位置，**则**算法结束；

否则切换当前位置的东邻方块为

新的当前位置；

}

否则 { … …

}

} **while (栈不空)** ；

若栈不空且栈顶位置尚有其他方向未被探索，
则设定新的当前位置为：沿顺时针方向旋转
找到的**栈顶位置的下一相邻块**；

若栈不空但栈顶位置的四周均不可通，
则｛删去栈顶位置；//从路径中删去该通道块
若栈不空，**则**重新测试新的栈顶位置，
直至找到一个可通的相邻块或出栈至栈空；
｝

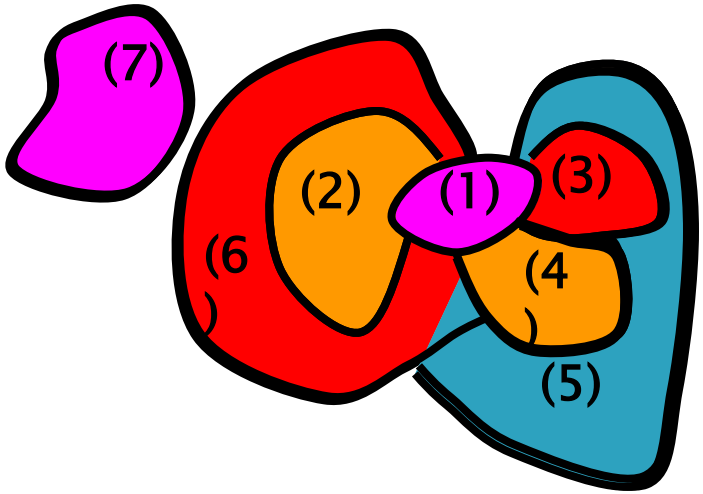
若栈空，则表明迷宫没有通路。



地图四染色问题

R [7][7]

	1	2	3	4	5	6	7	
1	0	1	1	1		1	1	0
2	1	0	0	0		0	1	0
3	1	0	0	1		1	0	0
4	1	0	1	0		1	1	0
5	1	0	1	1		0	1	0
6	1	1	0	1		1	0	0
7	0	0	0	0		0	0	0



1# 紫色

2# 黄色

3# 红色

4# 绿色

1	2	3	4	5	6	7
1	2	3	2	4	3	1

例五、 表达式求值

限于二元运算符的表达式定义:

表达式 ::= (操作数) + (运算符) + (操作数

操作数 ::= 简单变量 | 表达式

简单变量 :: = 标识符 | 无符号整数

表达式的三种标识方法：

设 Exp = S1 + OP + S2

则称 OP + S1 + S2 为**前缀表示法**

S1 + OP + S2 为**中缀表示法**

S1 + S2 + OP 为**后缀表示法**

例如: Exp = a × b + (c − d / e) × f

前缀式: + × a b × − c / d e f

中缀式: a × b + c − d / e × f

后缀式: a b × c d e / − f × +

结论: 5)后缀式的运算规则为:

3) 运算符在式中出现的顺序恰为运算的先后次序(先序不变),

4)前缀式的运算规则为:

连续出现的两个操作数和在它们之前且紧靠它们的运算符构成一个最小表达式,

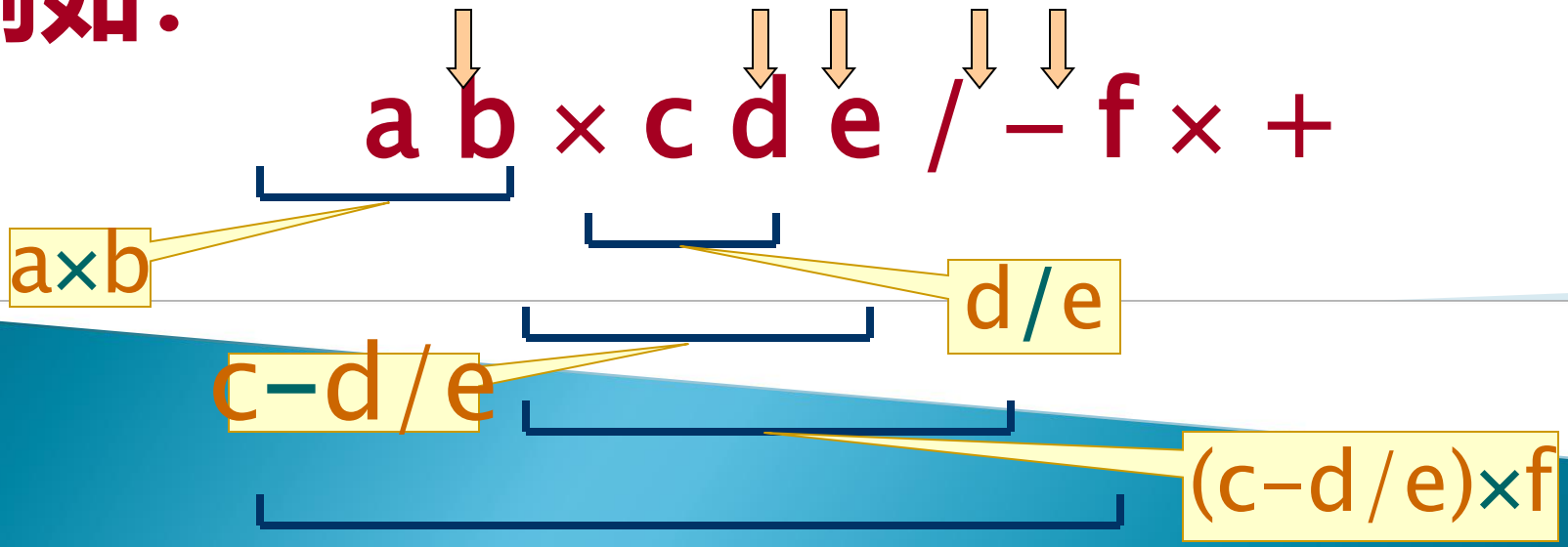
2) 运算符在式中出现时,已无括号,

将每个运算符和在它之前出现且紧靠它的两个操作数构成一个最小表达式。

如何从后缀式求值？

先找运算符，
再找操作数

例如：



用栈计算后缀表达式

第 44 页演示了后缀式的运算顺序，第 45–50 页讲中缀式如何转换为后缀式。补充操作数栈的求值实现。每个操作数或运算符是一个记号，支持二元 +、-、*、/。函数返回 1 表示成功，返回 0 表示表达式或运算无效。

```
#include <math.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

int eval_postfix(const char *const tokens[], size_t count, double *result)
{
    if (count == 0) {
        return 0;
    }
    double stack[count];
    size_t top = 0;
    for (size_t i = 0; i < count; ++i) {
        const char *token = tokens[i];
        char op = token[0];
        if (op && token[1] == '\0' && strchr("+-*/", op)) {
            if (top < 2) {
                return 0;
            }
            double right = stack[--top];  // 减法和除法先弹右操作数
            double left = stack[--top];
            double value;
            switch (op) {
            case '+': value = left + right; break;
            case '-': value = left - right; break;
```

用栈计算后缀表达式

```
            case '*': value = left * right; break;
            default:
                if (right == 0.0) {
                    return 0;
                }
                value = left / right;
                break;
            }
            if (!isfinite(value)) {
                return 0;
            }
            stack[top++] = value;
        } else {
            char *end;
            double value = strtod(token, &end);
            if (end == token || *end != '\0' || !isfinite(value)) {
                return 0;
            }
            stack[top++] = value;
        }
    }
    if (top != 1) {
        return 0;
    }
    *result = stack[0];
    return 1;
}
```

如何从原表达式求得后缀式?

分析“原表达式”和“后缀式”中的运算

原表达式: a + b × c – d / e × f

后缀式: a b c × + d e / f × –

每个运算符的运算次序要由**它之后的一个运算符**来定，在后缀式中，优先数高的运算符领先于优先数低的运算符。

从原表达式求得后缀式的规律为

1）设立**运算符栈；**

2）设表达式的结束符为“#”，
予设运算符栈的栈底为“#”

3）若当前字符是操作数，
则直接发送给后缀式。

从原表达式求得后缀式的规律为

4）若当前运算符的优先数高于栈顶运算符，则进栈；

5）否则，退出栈顶运算符发送给后缀式；

6）“(”对它之前后的运算符起隔离作用，“)”可视为自相应左括弧开始的表达式的结束符。

算符间的优先关系

θ2

θ1	+	−	×	/	(	)	#
+	>	>	<	<	<	>	>
−	>	>	<	<	<	>	>
×	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=

说明：

- 左括号 ‘(’的优先级最高
- 当θ1 = θ2 时，令

  θ1 >θ2。（左结合性）
- 为了算法处理的方便，在表达式的最左和最右边各添加一个 ‘#’。
- 优先关系相等的情况只有 ‘(’=‘)’和 ‘#’=‘#’

```
void transform(char suffix[ ], char exp[ ]
) {
  InitStack(S);  Push(S, '#');
  p = exp;  ch = *p;
  while (!StackEmpty(S)) {
      if (!IN(ch, OP)) Pass( Suffix, ch);
      else {              }
      if ( ch!= '#' ) { p++;  ch = *p; }
      else { Pop(S, ch);  Pass(Suffix, ch); }
  } // while
```



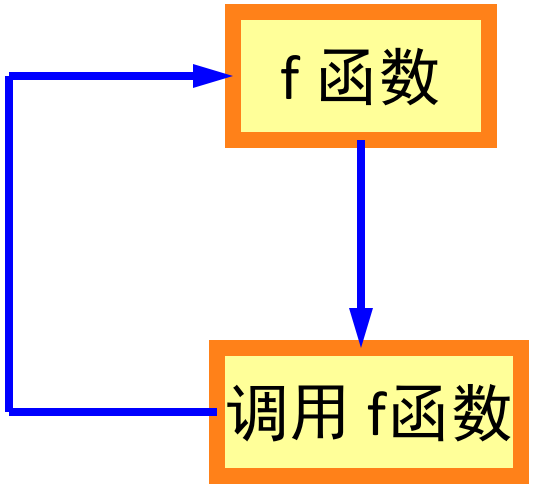
```
switch (ch) {
   case '(' :  Push(S, ch); break;
   case ')' :  Pop(S, c);
                  while (c!= '(' )
                     { Pass( Suffix, c);  Pop(S,
c) }
                  break;
    defult  :
       while(Gettop(S, c) && (
precede(c,ch)))
          { Pass( Suffix, c);  Pop(S, c); }
       if ( ch!= '#' )  Push( S, ch);
```

例六、实现递归

1. 定义：在调用一个函数的过程中直接或间接地调用该函数本身，称为递归函数。

- 直接调用

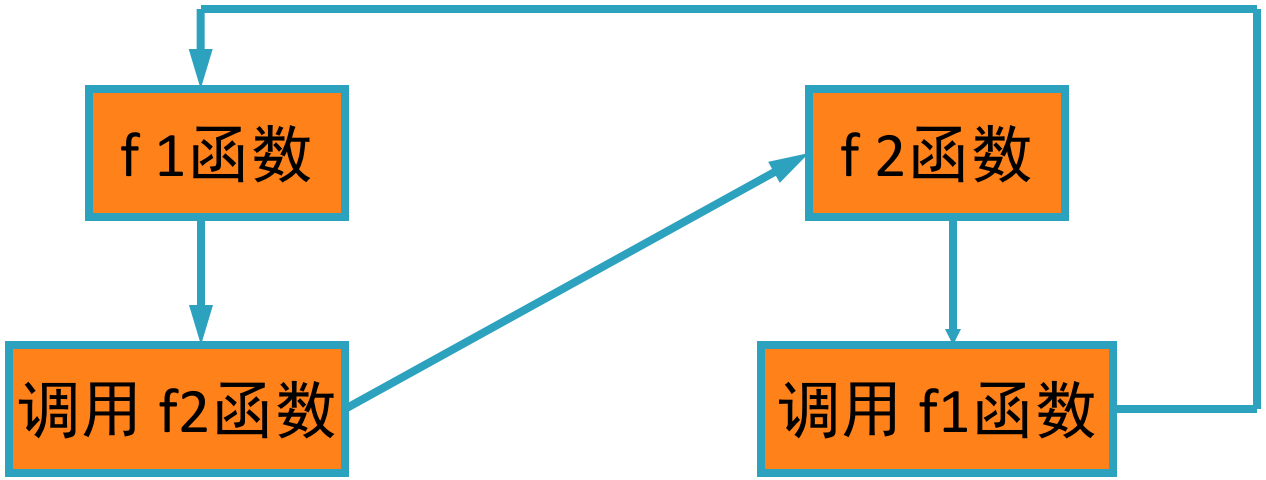
```
int f(x)
int x;
{   int y,z;
     …..
    z=f(x);
    ……
    return (2*z);
}
```



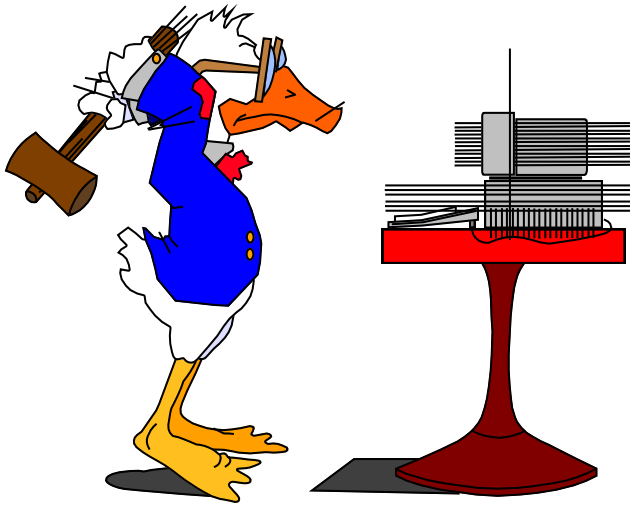
间接调用

```
int f1(x)
int x;
{   int y,z;
     .....
    z=f2( y);
    ......
 return (2*z);  }
```

```
int f2(t)
int t;
{   int a,c;
     .....
    c=f1(a);
    ......
 return (3+c);  }
```



- 递归表现为：在该函数的所有可能执行路径中，存在一条由于调用自身或其它函数所导致的环路路径；为确保函数最终在有限的时间内执行完毕，必须在环路中存在一个出口，即当某种条件成立时，不必执行环路，而直接执行一条通向结束的非环路线。



栈与递归的实现

- 递归的对象：一个对象部分地包含它自己，或用它自己给自己定义。如某些数据结构的定义

  线性表的另一种定义:

  - 若元素个数为0，则称为空表
  - 若元素个数大于0，则有且仅有一个第一个元素(即表头)，剩余元素形成一个表(即表尾) 。

- 递归的过程：一个过程直接或间接地调用自己

  如：0的阶乘是1，n(n>0)的阶乘等于n乘上(n-1)的阶乘

递归的应用

- 递归定义：如阶乘、Fibonacci数列等
- 数据结构：如表、树、二叉树等
- 问题求解：如Hanoi塔、八皇后



栈与递归的实现-阶乘函数

- 定义是递归的

$$n! = \begin{cases} 1, & \text{当} n = 0 \text{时} \\ n * (n-1)!, & \text{当} n \geq 1 \text{时} \end{cases}$$

求解阶乘函数的递归算法

```
long fact ( long n ) {
    if ( n == 0 ) return 1;              //递归结束条件
    else return n * fact (n-1);          //递归的规则
}
```

栈与递归的实现-阶乘函数



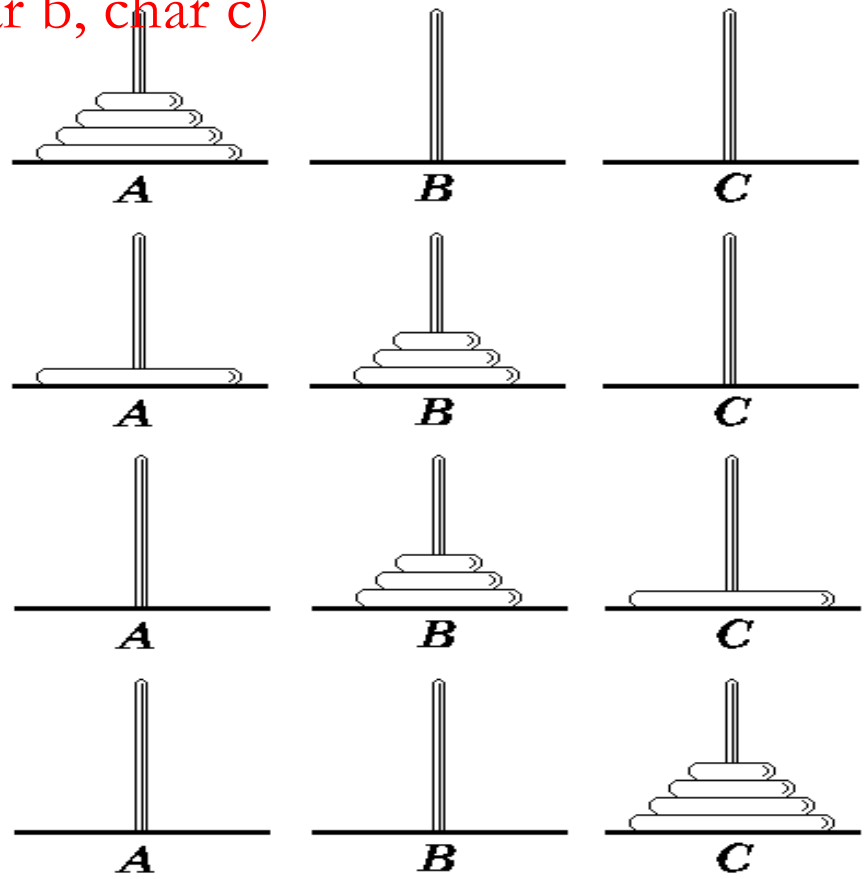
栈与递归的实现-数据结构是递归的

- 数据结构是递归的--表
  - 空表
  - 非空表: (表头元素+除表头元素以外的剩余元素)
  - 查找表L中是否有元素值e

```
LinkList search(LinkList L, ElemType e)
// L为不带头结点的单向非循环链表
{
   if（L==NULL）return NULL;
   else if（L->data==e）return L;
   else return search(L->next, e);
}
```

栈与递归的实现-问题求解是递归的

- Hanoi塔问题
- void hanoi(int n, char a, char b, char c)
  n-圆盘数　a-源塔座
  b-中介塔座 c-目标塔座
- 搬动方法
  - n=1, a->c
  - n>1:
    hanoi(n-1, a, c, b)
    a->c
    hanoi(n-1, b, a, c)
- 注意
  用递归调用的结果，不关注该结果如何获得的细节



栈与递归的实现-递归实现

- 调用函数与被调用函数---系统工作栈
  - 执行被调用函数前
    - 现场保护：实在参数、返回地址
    - 为被调用函数的局部变量分配存储区
    - 将控制转移到被调函数的入口
  - 从被调用函数返回调用函数前
    - 保存被调函数的计算结果
    - 释放被调函数的数据区
    - 现场恢复：返回

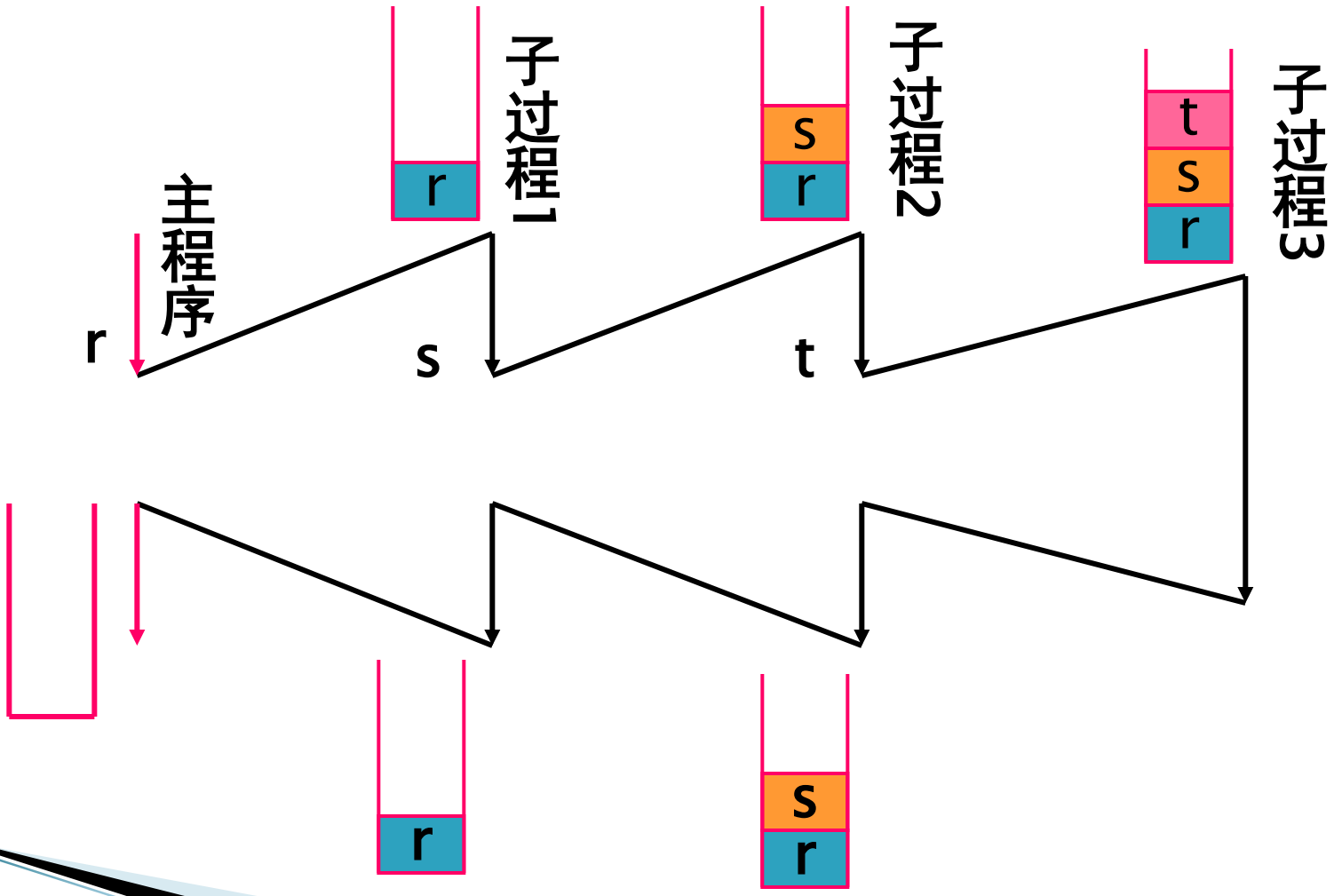


栈与递归的实现-递归实现

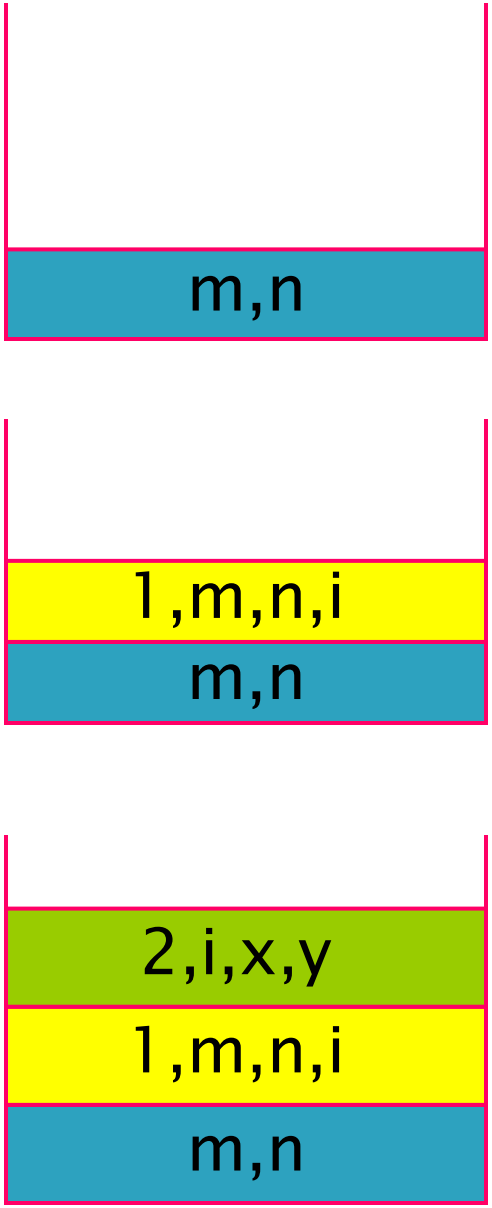
- 递归过程与递归工作栈
  实际的系统中，一般综合考虑递归调用和非递归调用，统一处理
- 递归工作栈
  - 活动记录(栈帧stack frame)
    实在参数、局部变量、上一层的返回地址
  - 每进入一层递归，产生一个新的工作记录入栈
  - 每退出一层递归，就从栈顶弹出一个工作记录
  - 当前执行层的工作记录必是栈顶记录
- 例子：　hanoi(3,a,b,c)



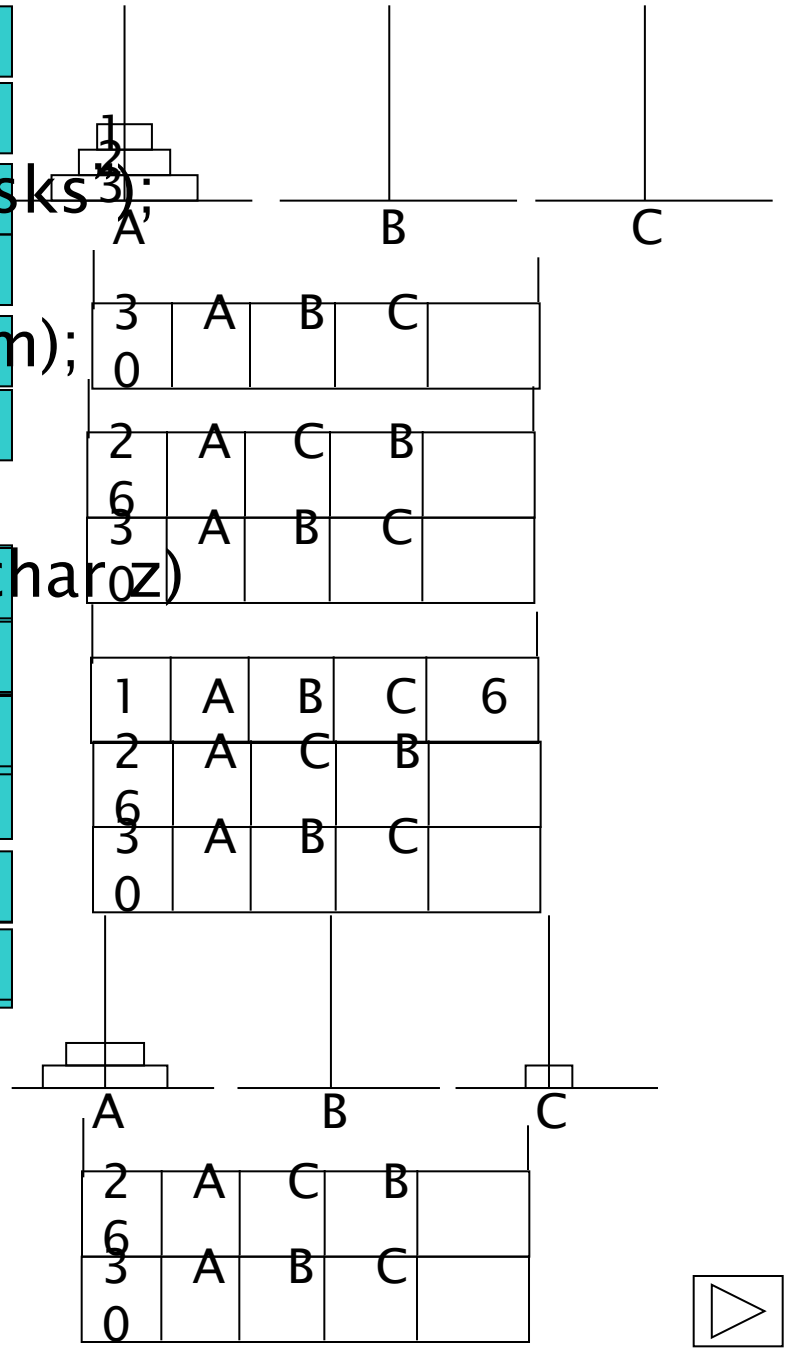
程序嵌套调用的示意图



```
Void main(){
Int m,n;
First(m,n);
1:……
}
Int first(int s,int t){
Int i;
Second(i);
2:….
}
Int second(int d){
Int x,y;
….}实现
```

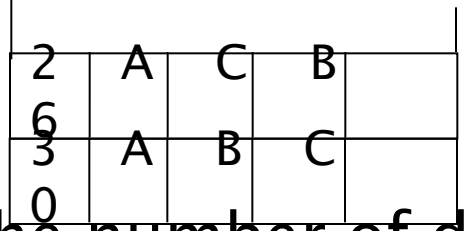


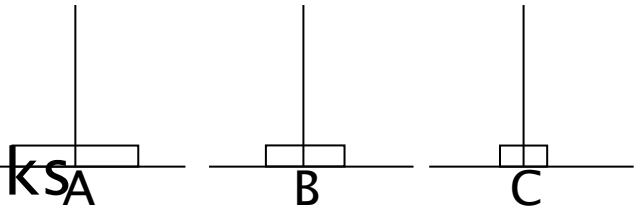
```
 main()
   {   int m;
       printf("Input number of disks");
       scanf("%d",&m);
       printf("Steps : %3d disks",m);
       hanoi(m,'A','B','C');
(0) }
void hanoi(int n,char x,char y,char z)
(1)  {
(2)     if(n==1)
(3)        move(1,x,z);
(4)     else{
(5)           hanoi(n-1,x,z,y);
(6)           move(n,x,z);
(7)           hanoi(n-1,y,x,z);
(8)        }
(9)  }
```

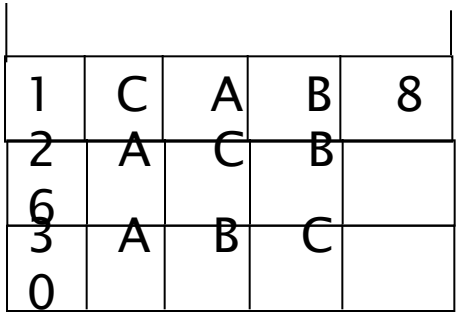


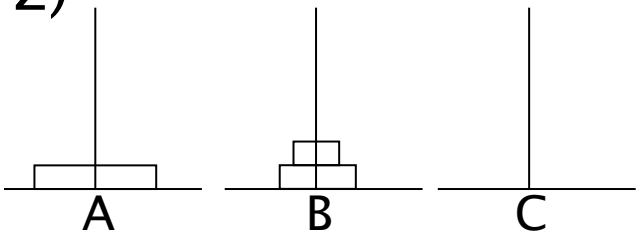
```
main()
  {   int m;
      printf("Input the number of disks:");
      scanf("%d",&m);
      printf("The steps to moving %3d disks:\n",m);
      hanoi(m,'A','B','C');
(0) }
```

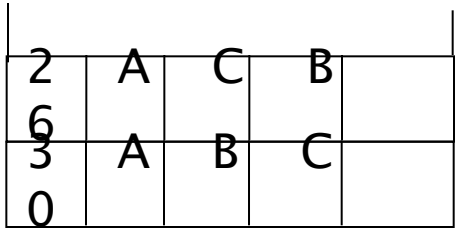
```
void hanoi(int n,char x,char y,char z)
(1)  {
(2)     if(n==1)
(3)        move(1,x,z);
(4)     else{
(5)           hanoi(n-1,x,z,y);
(6)           move(n,x,z);
(7)           hanoi(n-1,y,x,z);
(8)        }
(9)  }
```

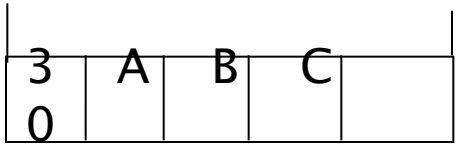






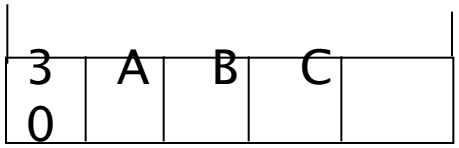


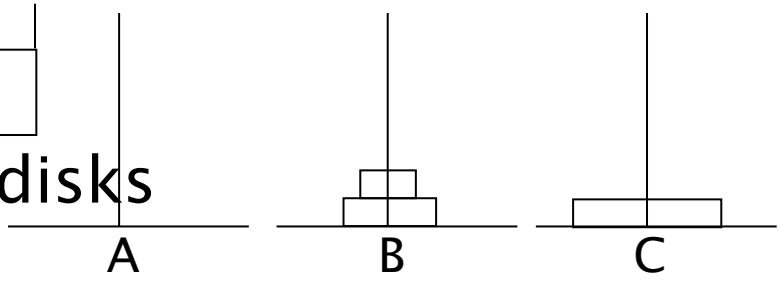


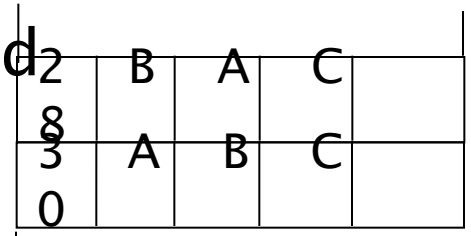


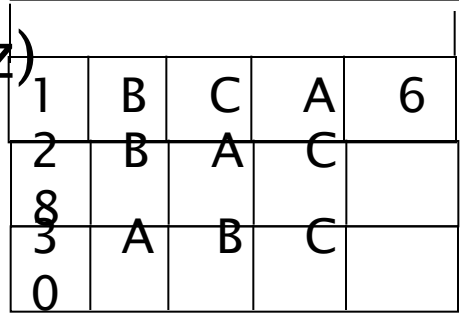
```
main()
  {   int m;
      printf("Input the number of disks
      scanf("%d",&m);
      printf("The steps to moving %3d
      hanoi(m,'A','B','C');
(0) }
```

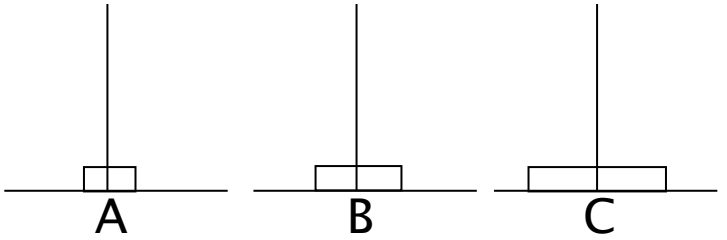
```
void hanoi(int n,char x,char y,char z)
(1)  {
(2)      if(n==1)
(3)          move(1,x,z);
(4)      else{
(5)              hanoi(n-1,x,z,y);
(6)              move(n,x,z);
(7)              hanoi(n-1,y,x,z);
(8)          }
(9)  }
```

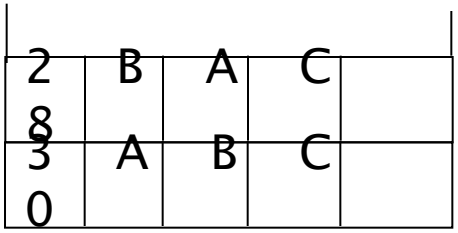




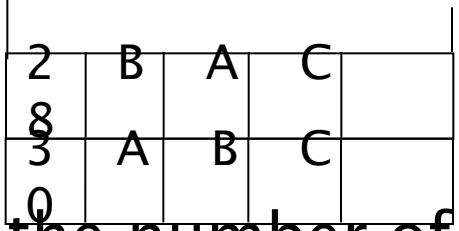


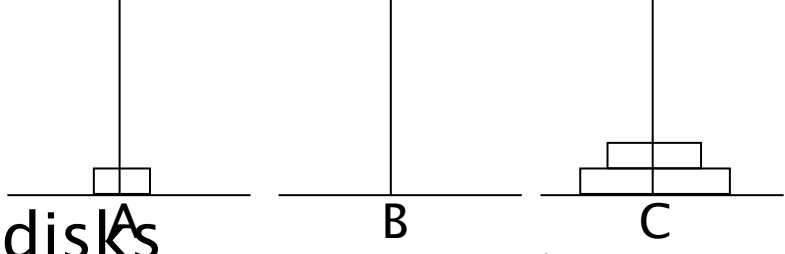


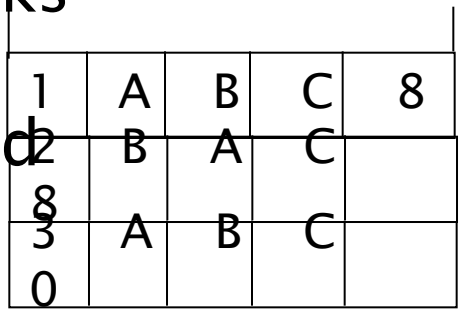


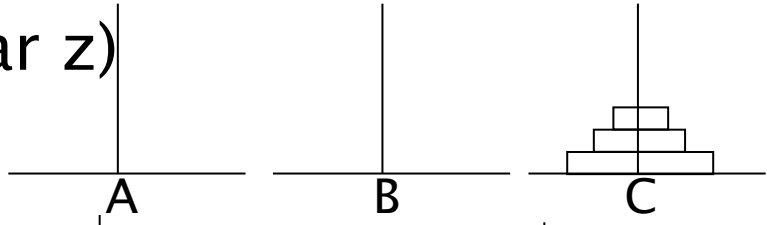


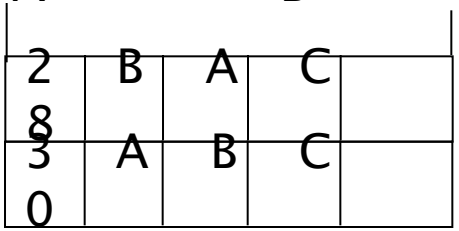
```
main()
  {   int m;
      printf("Input the number of disks
      scanf("%d",&m);
      printf("The steps to moving %3d
      hanoi(m,'A','B','C');
(0) }
void hanoi(int n,char x,char y,char z)
(1)  {
(2)     if(n==1)
(3)        move(1,x,z);
(4)     else{
(5)           hanoi(n-1,x,z,y);
(6)           move(n,x,z);
(7)           hanoi(n-1,y,x,z);
(8)        }
(9)  }
```

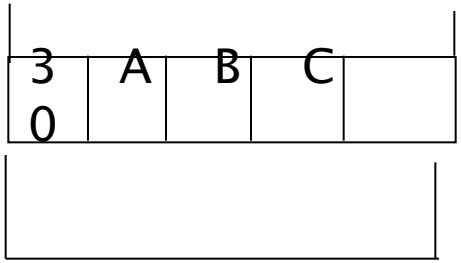




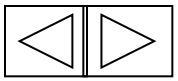








栈空





Hanoi.c D:\fengyi\bi power\power.c

3.4 队列的类型定义

ADT Queue {

数据对象：

$$D=\{a_i \mid a_i \in ElemSet,\ i=1,2,\ldots,n,$$

数据关系：

$$R1=\{ <a_{i-1},a_i> \mid a_{i-1}, a_i \in D,\ i=$$

约定其中a_1 端为**队列头**，a_n 端为**队列尾**

基本操作：

} ADT Queue

队列的基本操作：

InitQueue(&Q)　　DestroyQueue(&Q)

QueueEmpty(Q)　　QueueLength(Q)

GetHead(Q, &e)　　ClearQueue(&Q)

EnQueue(&Q, e)　　DeQueue(&Q, &e)

QueueTravers(Q, visit())



InitQueue(&Q)

操作结果：**构造一个空队列Q。**

DestroyQueue(&Q)

初始条件：**队列Q已存在。**

操作结果：**队列Q被销毁，不再存在。**



QueueEmpty(Q)

初始条件：**队列Q已存在。**

操作结果：**若Q为空队列，则返回TRUE，否则返回FALSE。**



QueueLength(Q)

初始条件：**队列Q已存在。**

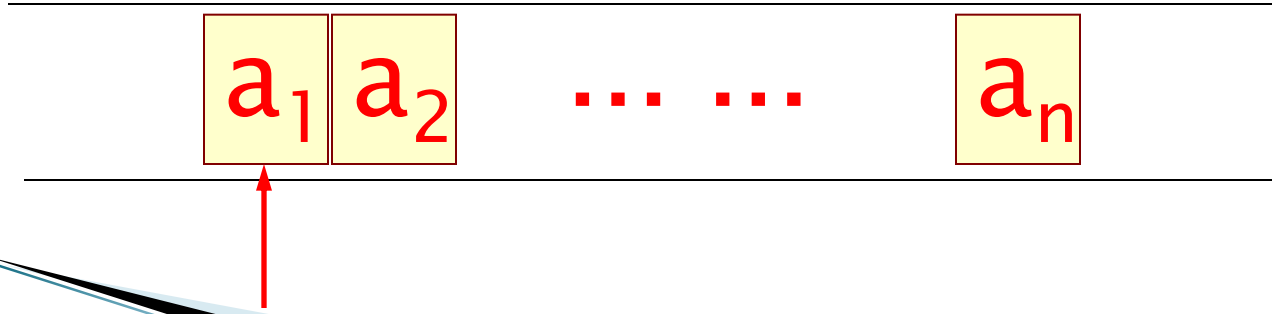
操作结果：**返回Q的元素个数，即队列的长度。**



GetHead(Q, &e)

初始条件：**Q为非空队列。**

操作结果：**用e返回Q的队头元素。**



ClearQueue(&Q)

初始条件：**队列Q已存在。**
操作结果：**将Q清为空队列。**



EnQueue(&Q, e)

初始条件：**队列Q已存在。**

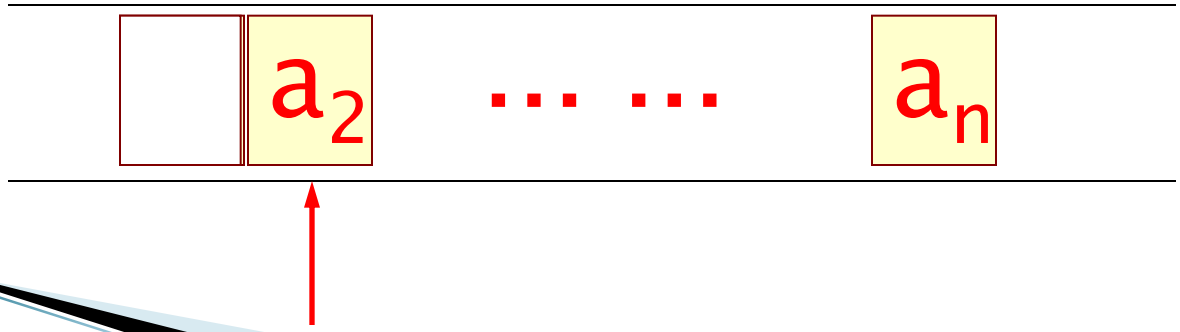
操作结果：**插入元素e为Q的新的队尾元素。**



DeQueue(&Q, &e)

初始条件：**Q为非空队列。**

操作结果：**删除Q的队头元素，并用e返回其值。**



3.5 队列类型的实现

链队列——链式映象

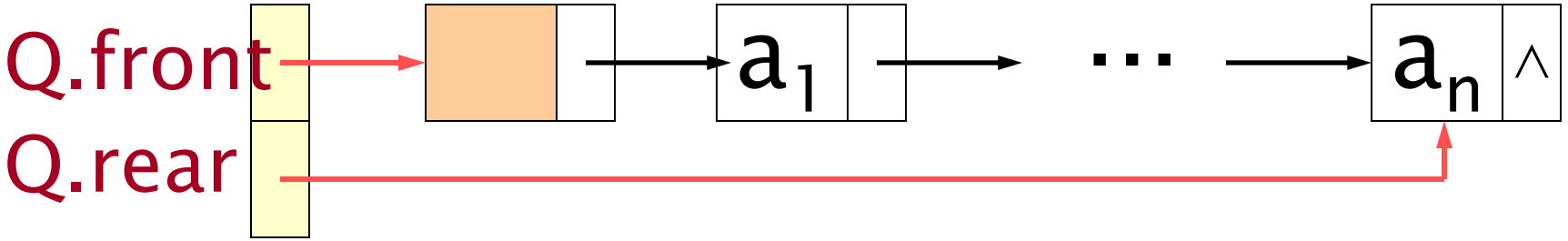
循环队列——顺序映象



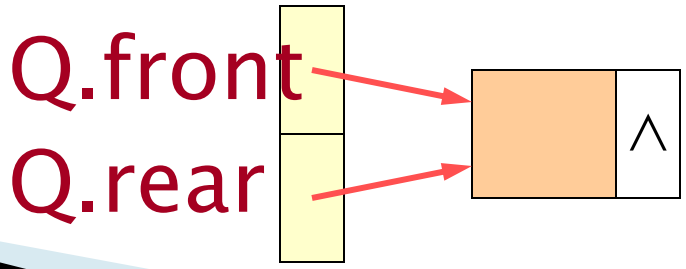
链队列——链式映象

```
typedef struct QNode {//结点类型
   QElemType      data;
   struct QNode   *next;
} QNode, *QueuePtr;
```

```
typedef struct { // 链队列类型
    QueuePtr  front;  // 队头指针
    QueuePtr  rear;   // 队尾指针
} LinkQueue;
```



空队列



```
Status InitQueue (LinkQueue &Q) {
  // 构造一个空队列Q
  Q.front = Q.rear =
(QueuePtr)malloc(sizeof(QNode));
  if (!Q.front) exit (OVERFLOW);
                                  //存储分配失败
  Q.front->next = NULL;
```

```
Status EnQueue (LinkQueue &Q,

QElemType e) {
    // 插入元素e为Q的新的队尾元素
    p = (QueuePtr) malloc (sizeof
(QNode));
    if (!p)  exit (OVERFLOW);    //存储分配
失败
    p->data = e;   p->next = NULL;
```

```
Status DeQueue (LinkQueue &Q,

QElemType &e) {
  // 若队列不空，则删除Q的队头元素，
  //用 e 返回其值，并返回OK；否则返回ERROR
   if (Q.front == Q.rear)    return
ERROR;
   p = Q.front->next;   e = p->data;
   Q.front->next = p->next;
   if (Q.rear == p)  Q.rear = Q.front;
```

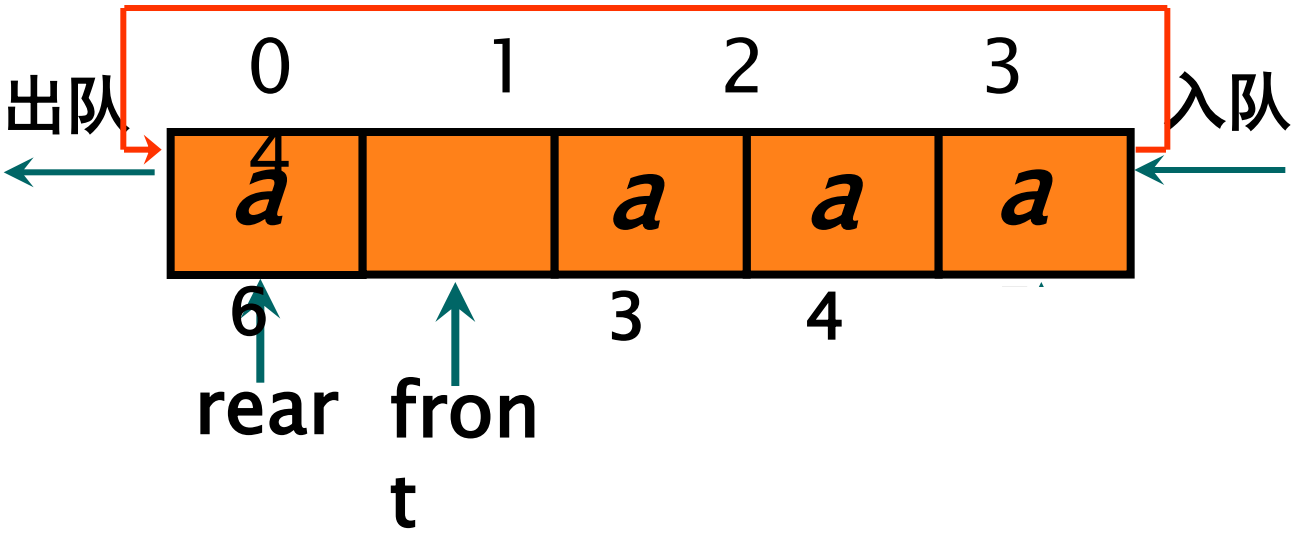


循环队列——顺序映象

```
#define MAXQSIZE  100  //最大队列长度
  typedef struct {
    QElemType  *base;  // 动态分配存储空
    int  front;      // 头指针，若队列不空，
                     //  指向队列头元素
    int  rear;       // 尾指针，若队列不空，指
                     // 队列尾元素 的下一个位
  } SqQueue;
```

队列的顺序存储结构及实现（顺序队列）

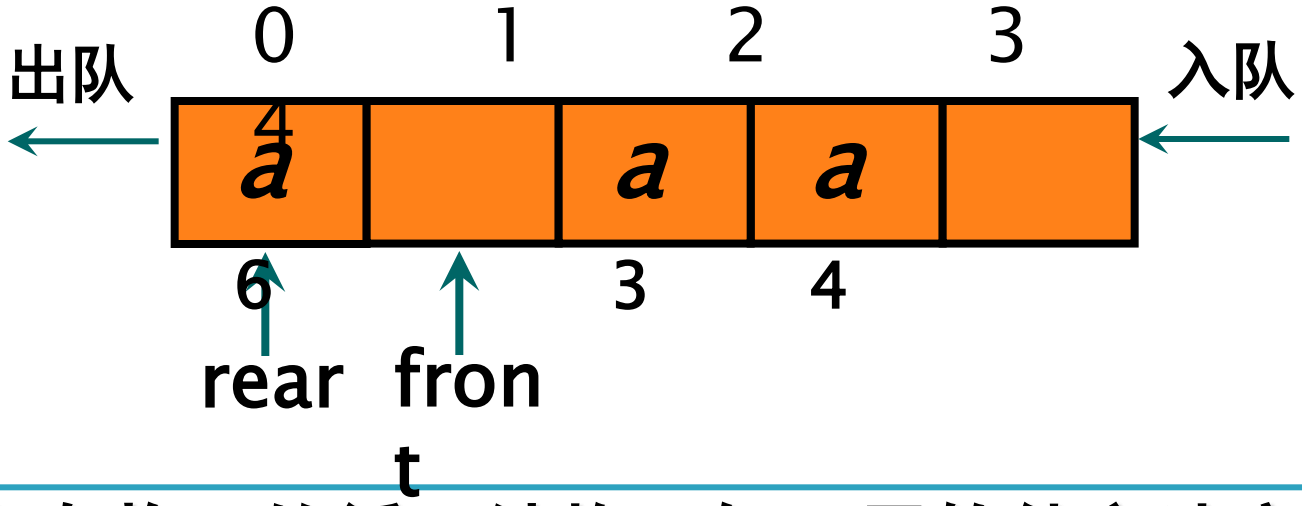
如何解决假上溢？



循环队列：将存储队列的数组头尾相接。

队列的顺序存储结构及实现（顺序队列）

如何实现循环队列？



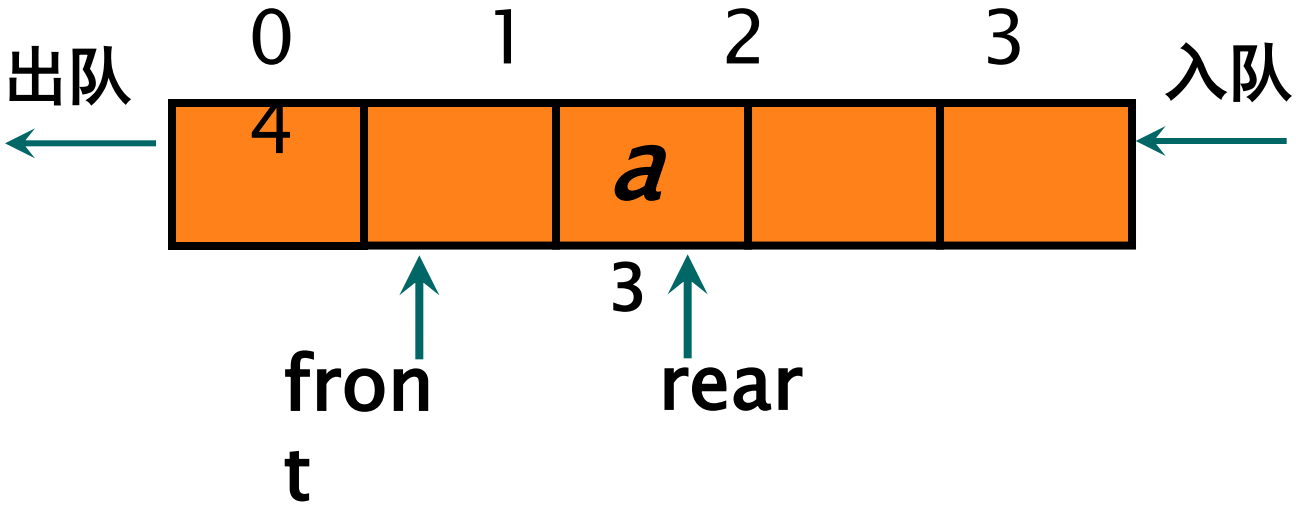
不存在物理的循环结构，但可用软件方法实现。

求模：（4＋1）mod 5＝0

队列的顺序存储结构及实现（顺序队列）

如何判断循环队列队空？

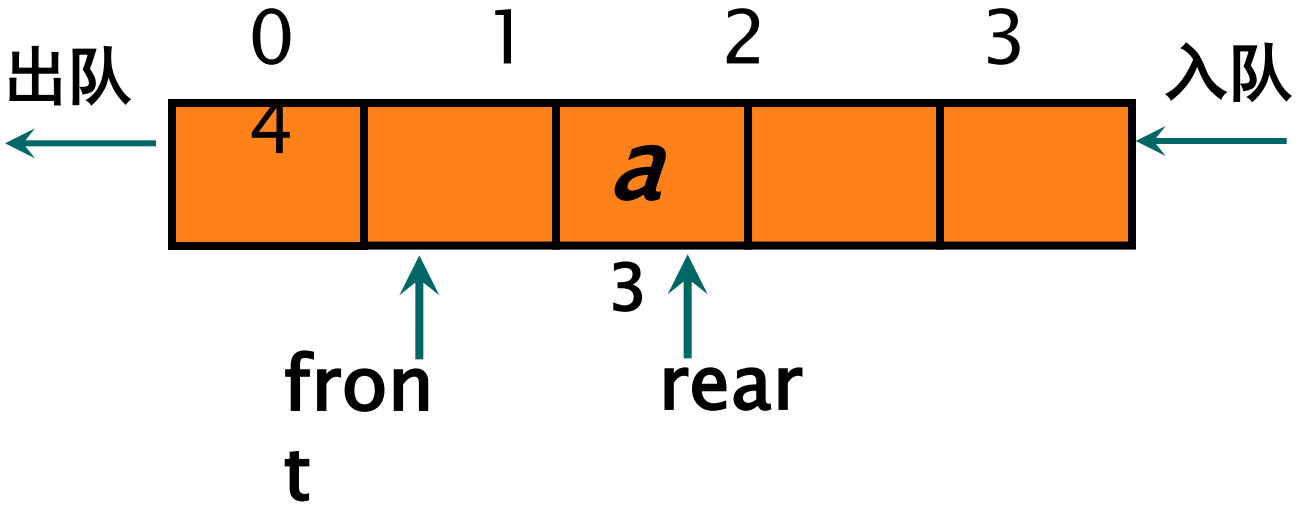
队空的临界状态



队列的顺序存储结构及实现（顺序队列）

如何判断循环队列队空？

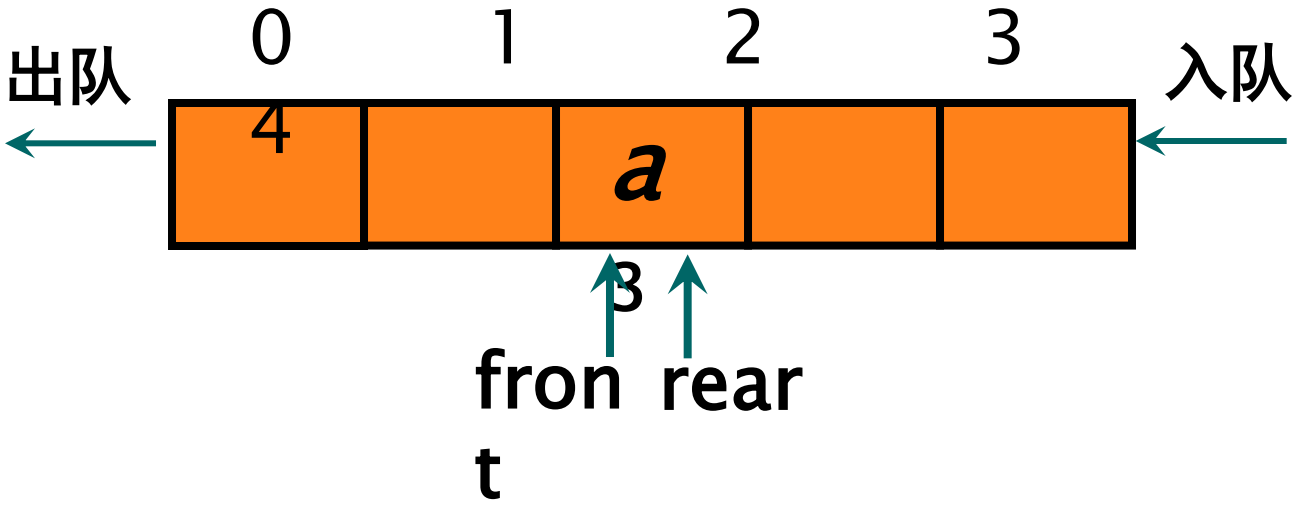
执行出队操作



队列的顺序存储结构及实现（顺序队列）

如何判断循环队列队空？

执行出队操作

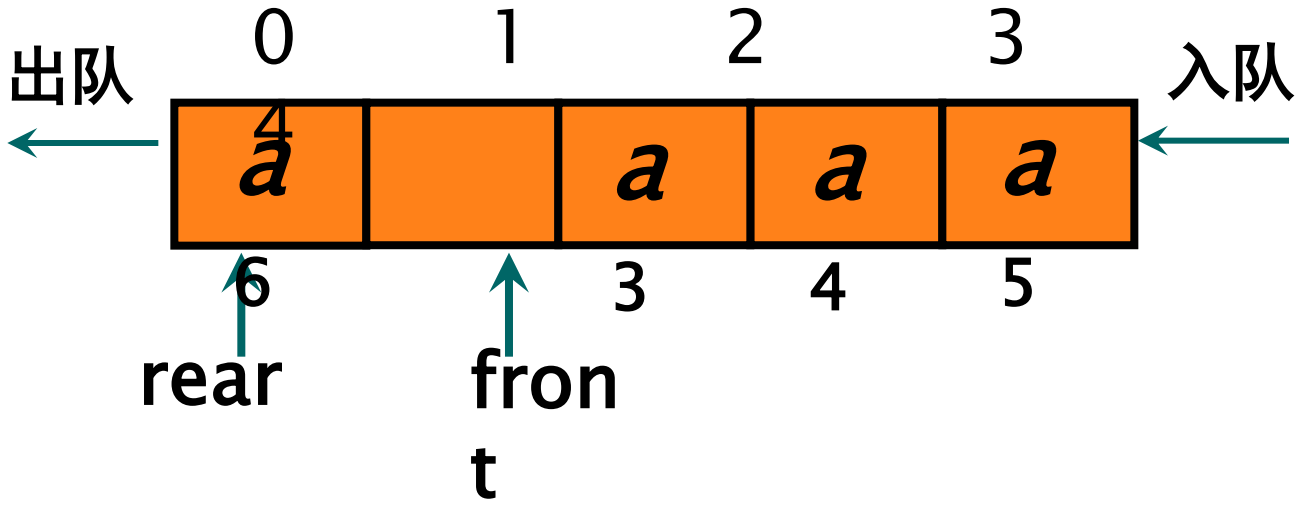


队空：
front=rear

队列的顺序存储结构及实现（顺序队列）

如何判断循环队列队满？

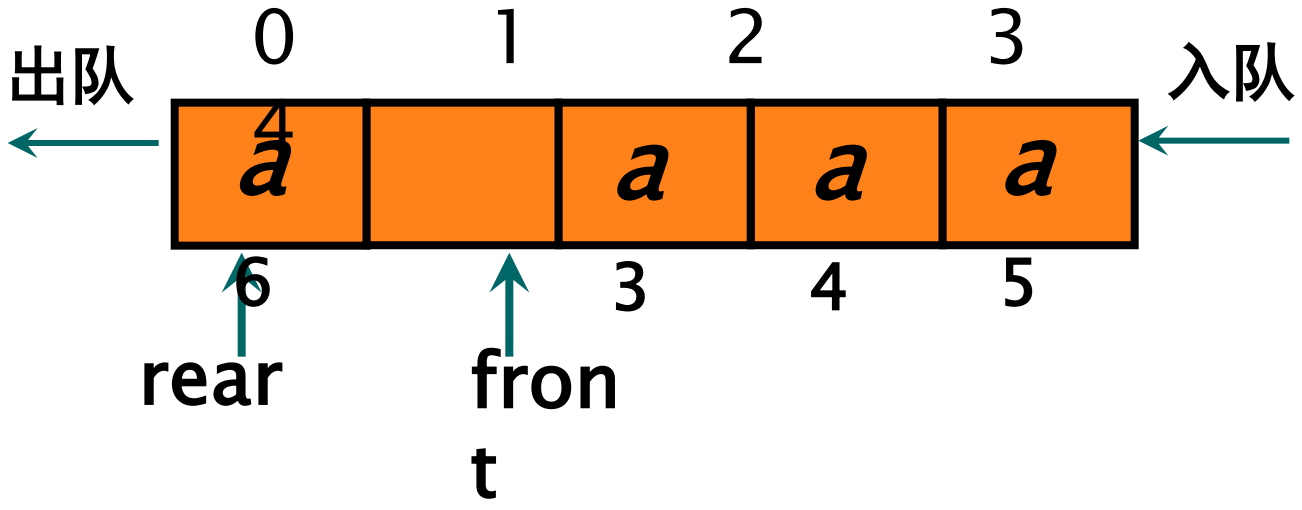
队满的临界状态



队列的顺序存储结构及实现（顺序队列）

如何判断循环队列队满？

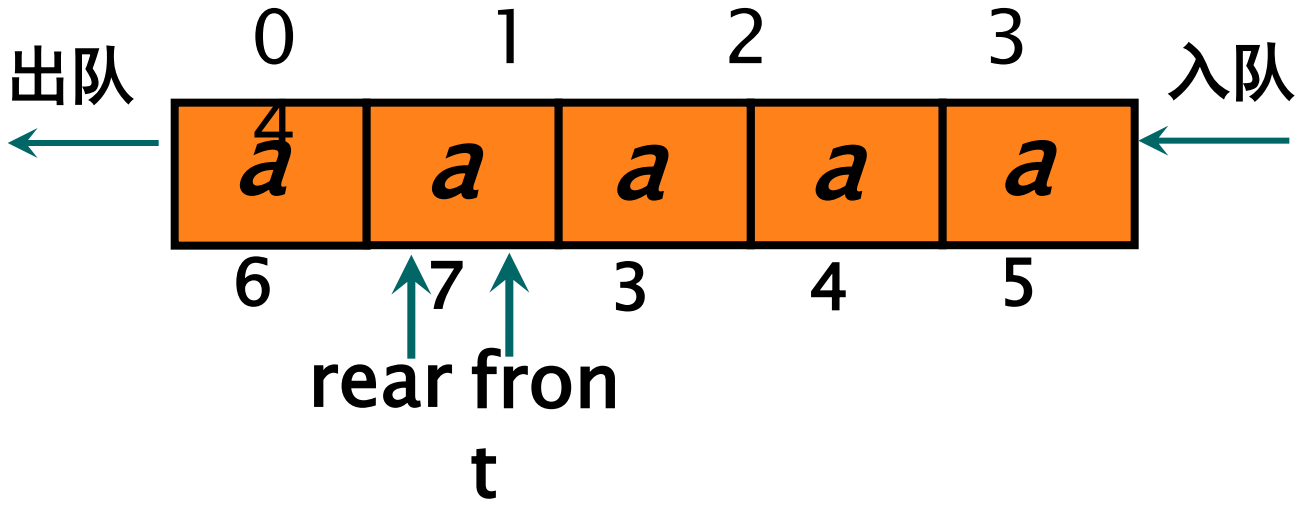
执行入队操作



队列的顺序存储结构及实现（顺序队列）

如何判断循环队列队满？

执行入队操作



队满：
front=rear

队列的顺序存储结构及实现（顺序队列）



队空、队满的判定条件出现二义性。
如何将队空和队满的判定条件分开？

方法一：修改队满条件，浪费一个元素空间，队满时数组中还有一个空闲单元；

方法二：附设一个存储队列中元素个数的变量num，当num=0时队空，当num=QueueSize时为队满；

方法三：设置标志flag，当front=rear且flag=0时为队空，当front=rear且flag=1时为队满。

```
Status InitQueue (SqQueue &Q) {
   // 构造一个空队列Q
   Q.base = (ElemType *) malloc
            (MAXQSIZE *sizeof
(ElemType));
    if (!Q.base) exit (OVERFLOW);
                                  // 存储
分配失败
    Q.front = Q.rear = 0;
```

```
Status EnQueue (SqQueue &Q,
ElemType e) {    // 插入元素e为Q的新的
队尾元素
    if ((Q.rear+1) % MAXQSIZE ==
Q.front)
        return ERROR; //队列满
    Q.base[Q.rear] = e;
    Q.rear = (Q.rear+1) % MAXQSIZE;
    return OK;
```

```
Status DeQueue (SqQueue &Q,
ElemType &e) {  // 若队列不空，则删除Q的队头元素，
   // 用e返回其值，并返回OK；否则返回ERROR
    if (Q.front == Q.rear)  return ERROR;
    e = Q.base[Q.front]; // 先读队头，再移动 front
    Q.front = (Q.front+1) % MAXQSIZE;
    return OK;
}
```



3.6 队列应用举例

- CPU资源的竞争问题。
- 主机与外部设备之间速度不匹配的问题（例如主机和打印机）
- 打印杨辉三角
- 划分子集问题

二项式系数值（杨辉三角）

第 1 行　　1　　1

第 2 行　　1　　2　　1

第 3 行　　1　　3　　3　　1

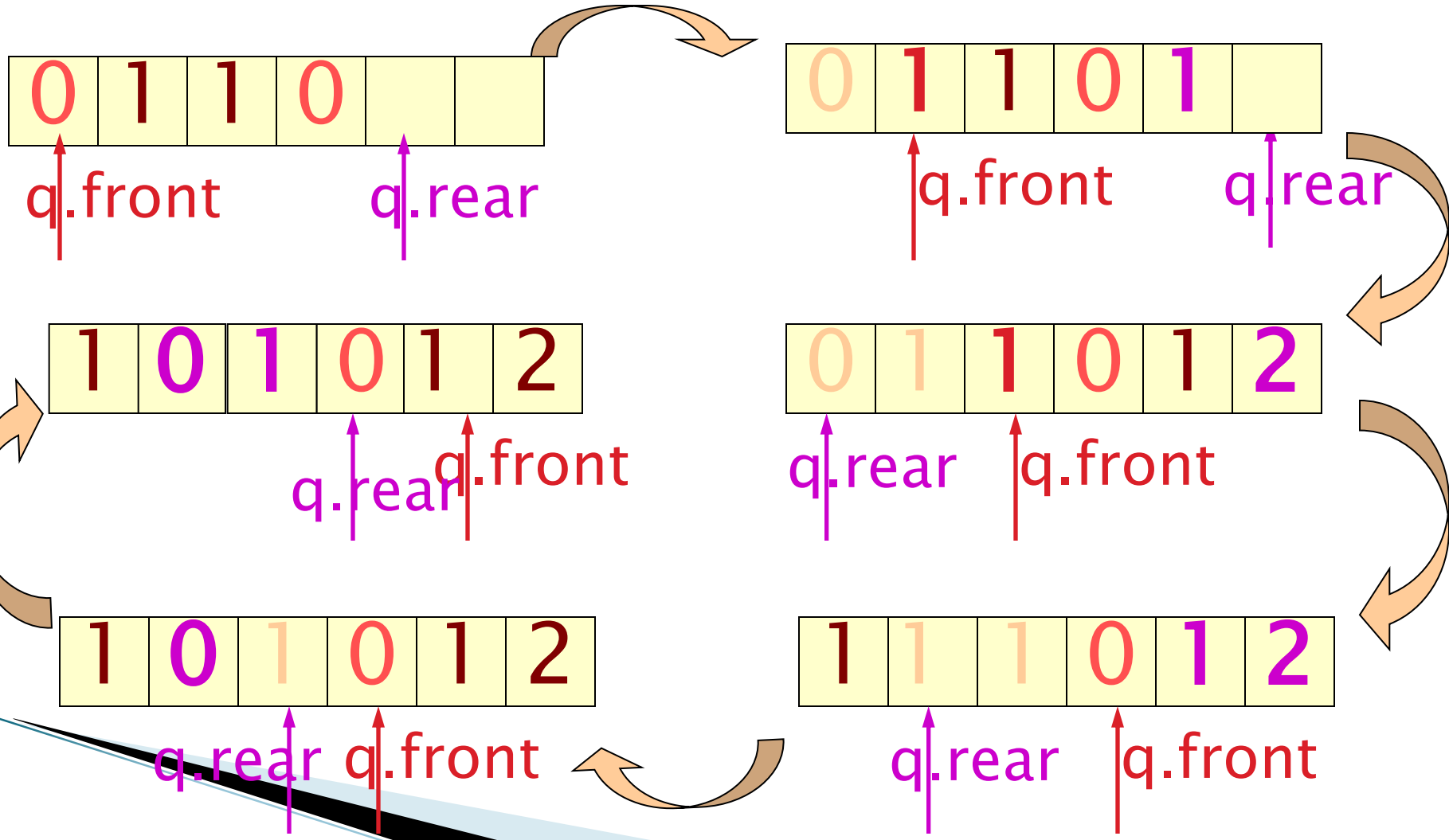
第 4 行　　1　　4　　6　　4　　1

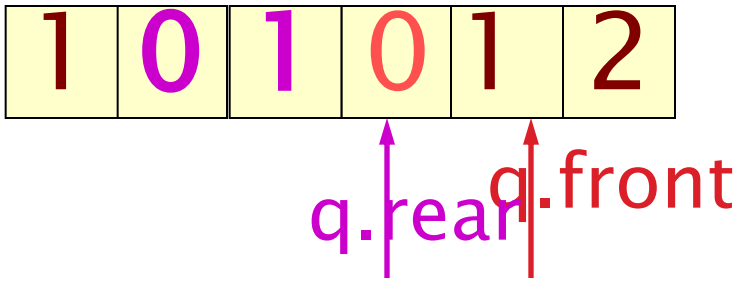
设第 i-1行的值：(a[0]=0) a[1]..a[i] (a[i+1]

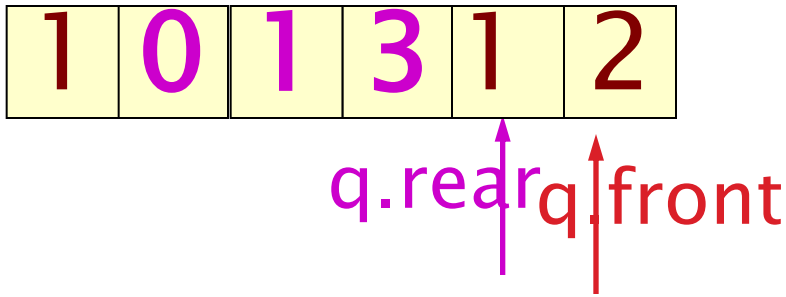
则第 i 行的值：b[j] = a[j-1]+a[j], j=1,2,...

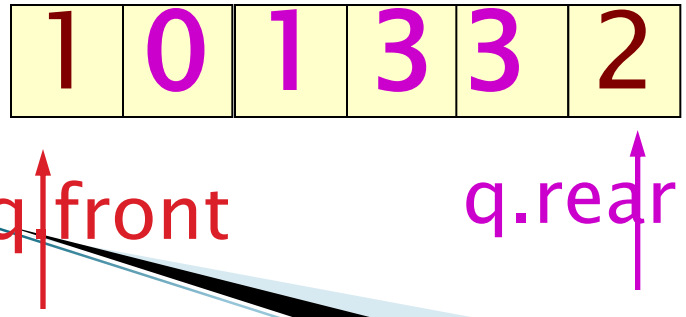
利用循环队列计算二项式的过程：

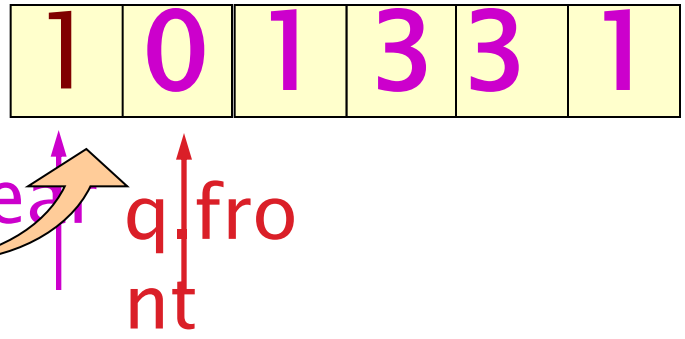
假设只计算三行，则队列的最大容量为 5。












```
do {
   DeQueue(Q, s);
   GetHead(Q, e);
   if (e!=0) printf ("%d", e);
   EnQueue(Q, s+e);
} while (e!=0);
```

程序如下：（n≤7）

```
#define MAXSIZE 10  /*定义队列的最大长度*/
#include <stdio.h>
#include <malloc.h>
typedef int datatype;
typedef struct
{           int data[MAXSIZE];
            int front;
            int rear;
}SeqQueue;

SeqQueue *InitQueue()  /*队列初始化*/
{ SeqQueue  *q;
q=(SeqQueue*)malloc(sizeof(SeqQueue));
q->front = q->rear = 0;
return q;
}
```

```
void EnQueue(SeqQueue *q, datatype x)   /*入队列*/
{ if((q->rear+1)%MAXSIZE==q->front)
    {printf("\n顺序循环队列是满的!");}
  else
    {q->data[q->rear] = x;
     q->rear = (q->rear+1)%MAXSIZE;
    }
}

datatype DeQueue (SeqQueue *q)    /*出队列*/
{datatype x;
 if(q->front==q->rear)
    { printf("\n顺序队列是空的！不能做删除操作！"); return 0;}
 x = q->data[q->front];
 q->front = (q->front+1)%MAXSIZE;
 return x;
}
```

```
int QueueEmpty(SeqQueue *q)   /*判队空*/
{ return(q->front==q->rear);
}

int GetHead(SeqQueue *q)   /*取队头元素*/
{int e;
  if(q->front==q->rear)    e=0;
  else   e=q->data[q->front];
  return e;
}

void YangHui( int n )     /*打印杨辉三角形的前n行, n≤7*/
{ SeqQueue  *q;
int i, j,s,t;
for(i=1;i<=n;i++)
printf("    ");
printf("1\n");      /*在中心位置输出杨辉三角最顶端的1(第0行)*/
q=InitQueue();      /*设置容量为n+2的空队列*/
EnQueue(q,0);      /*添加行分隔符*/
EnQueue(q,1);EnQueue(q,1);      /*第1行的值入队*/
```

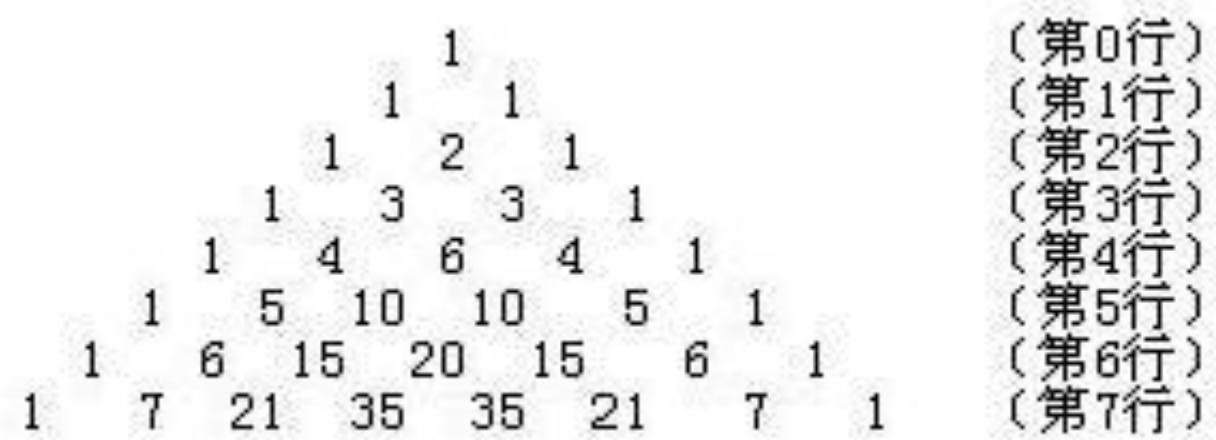
```
for(j=1;j<n;j++)      /*利用循环队列输出前n-1行的值*/
{ for(i=1;i<n-j;i++)      /*在输出第j行的首元素之间输出n-j个空格*/
     printf("   ");
  EnQueue(q,0);      /*行分隔符0入队*/
  do                  /*输出第j行并计算第j+1行*/
    {s=DeQueue(q);      /*删除队头元素并赋给s*/
     t=GetHead(q);     /*取队头元素给t*/
     if(t) printf("%5d",t);      /*若不到行分隔符则输出t，再输出一个空格*/
      else printf("\n");      /*否则输出一个换行符*/
     EnQueue(q,s+t);      /*将第j+1行的对应元素s+t入队*/
    }while(t!=0);
  }
  DeQueue(q);      /* 删除行分隔符 */
  printf("%3d",DeQueue(q)); /* 输出第n行的第一个元素 */
  while(!QueueEmpty(q))      /* 输出第n行的其余元素 */
   {t=DeQueue(q);
    printf("%5d",t);
   }
}
```

```
void main()
{int n;
 printf("\n请输入杨辉三角形的行数:\n");
 scanf("%d",&n);
 YangHui(n);
}
```

运行情况:

请输入杨辉三角形的行数: 7↓

输出结果:



队列的其他应用

- 优先级队列
- 离散时间模拟
- 图的广度优先遍历
- 基数排序

本章学习要点

1. 掌握栈和队列类型的特点，并能在相应的应用问题中正确选用它们。

2. 熟练掌握栈类型的两种实现方法，特别应注意栈满和栈空的条件以及它们的描述方法。

3. 熟练掌握循环队列和链队列的基本操作实现算法，特别注意队满和队空的描述方法。

4. 理解递归算法执行过程中栈的状态变化过程。

